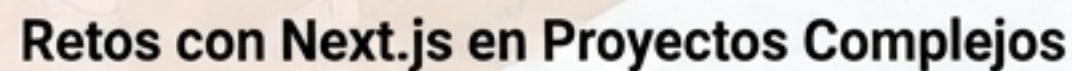


Aplicación práctica de 6 patrones de diseño en un stack moderno con Next.js y Supabase.

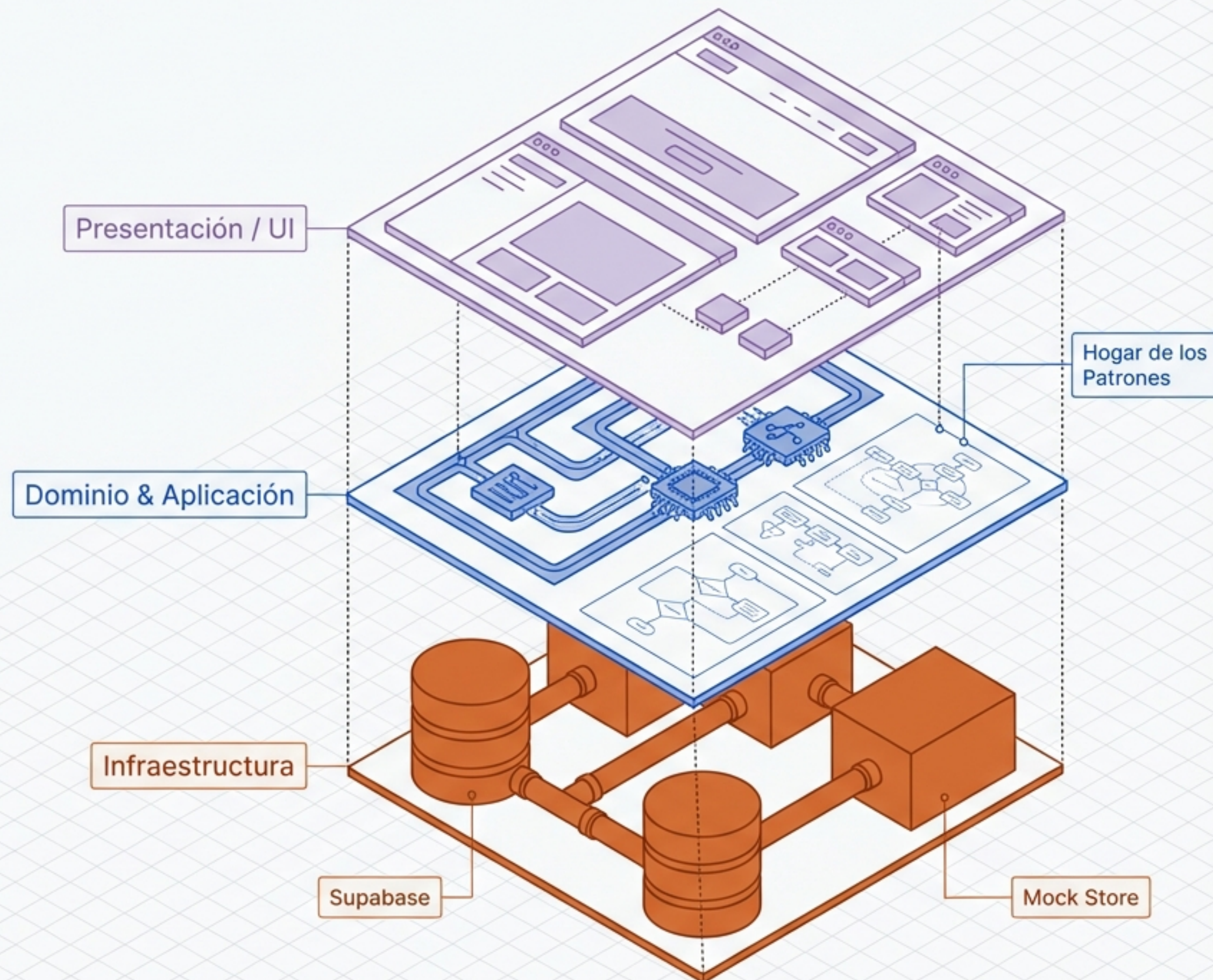
Aplicación práctica de 6 patrones de diseño en un stack moderno con Next.js y Supabase.



- Gran cantidad de clases y componentes que se deben crear.
- Dificultad en el entendimiento del código al utilizar múltiples patrones de diseño.

Arquitectura en Capas y Topología Visual

Taskflow aísla las reglas de negocio de la interfaz y la base de datos. Los patrones de diseño no son decorativos; habitan en el Dominio para orquestar la comunicación de forma estructurada.

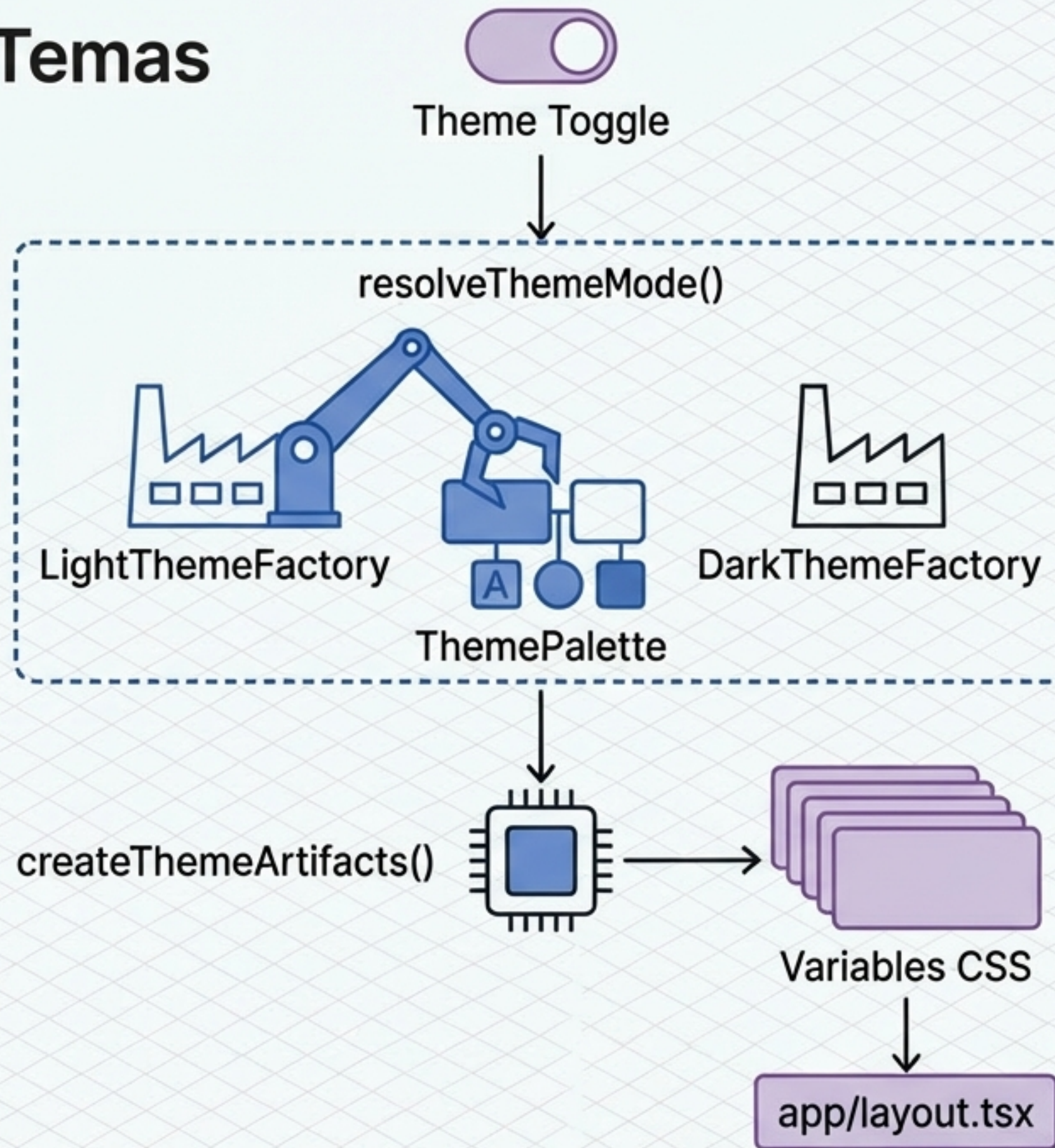


El Catálogo de Patrones de Taskflow

	Patrón	Categoría	Problema que resuelve	Implementación Real
	Abstract Factory	Creacional	Coherencia visual por tema	theme-factory.ts
	Singleton	Creacional	Única fuente de verdad global	theme-singleton.ts
	Builder	Creacional	Construcción de agregados complejos	task-builder.ts
	Factory Method	Creacional	Delegación según contexto	task-factory.ts
	Prototype	Creacional	Clonación con overrides técnicos	clone.ts
	Observer	Comportamiento	Desacoplar comandos de side-effects	project-event-publisher.ts

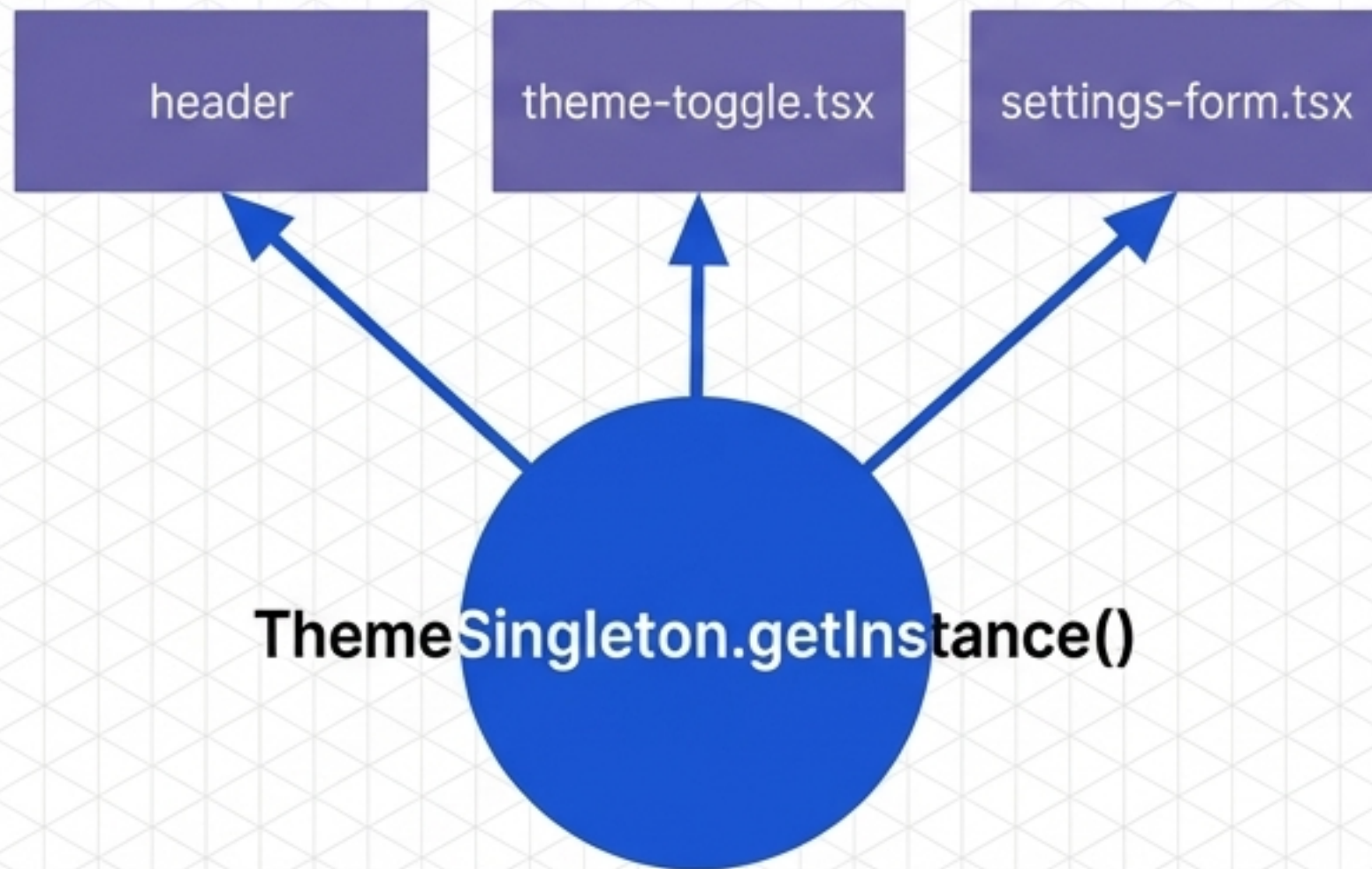
Abstract Factory: El Motor de Temas

No basta con cambiar un color suelto. Cada tema exige una familia coherente (fondo, borde, tipografía, acento). La fábrica encapsula la creación de la paleta entera, entregando artefactos visuales listos sin usar condicionales en la interfaz.



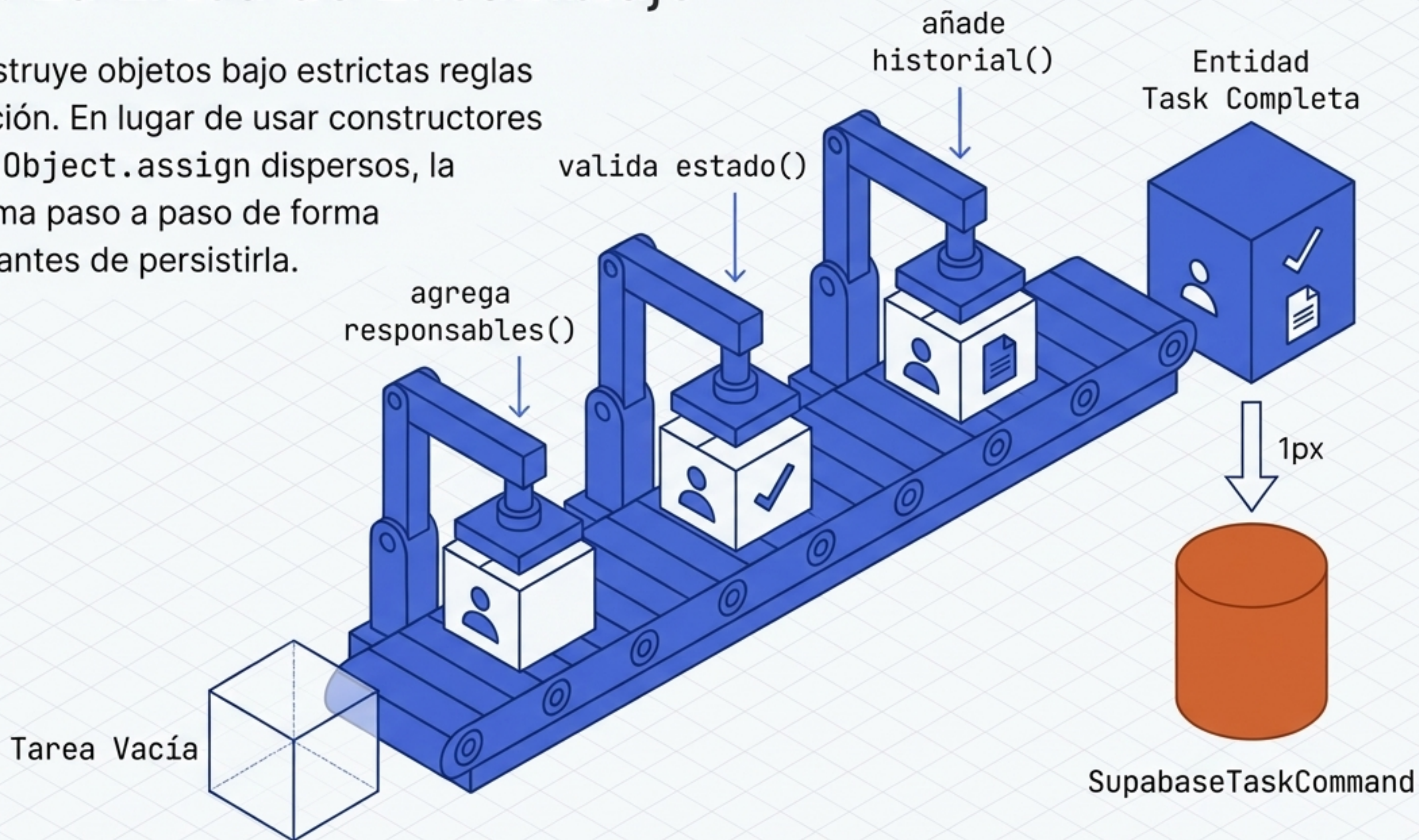
Singleton: La Única Fuente de Verdad

Evita la inconsistencia en el runtime manteniendo una única instancia compartida para el tema global del cliente y para la base de datos en memoria, asegurando que todos los componentes reaccionen al mismo estado.



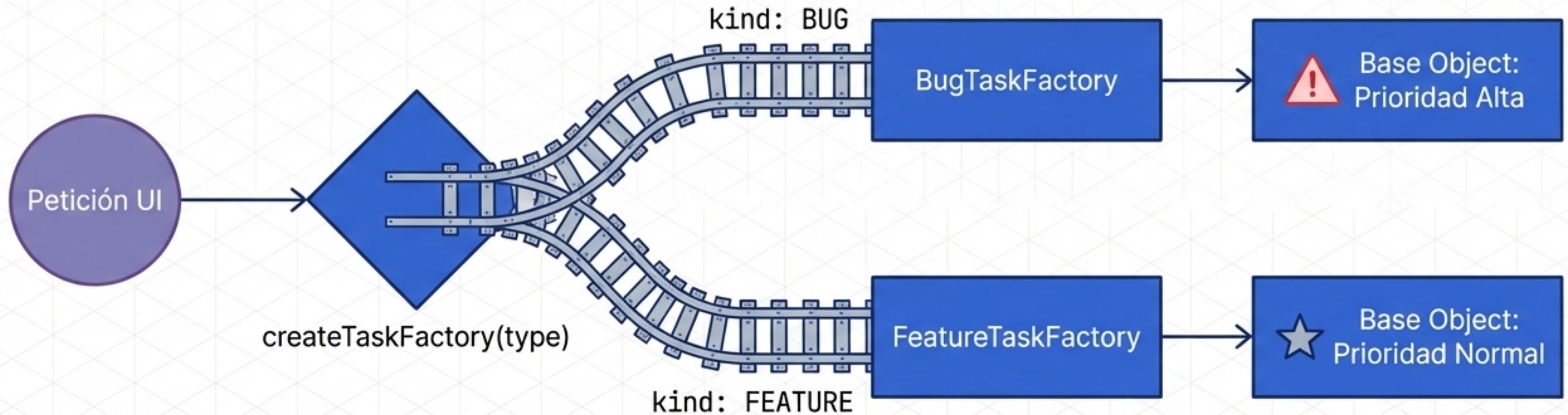
Builder: La Línea de Ensamblaje

Taskflow construye objetos bajo estrictas reglas de normalización. En lugar de usar constructores monolíticos u `Object.assign` dispersos, la entidad se arma paso a paso de forma transparente antes de persistirla.

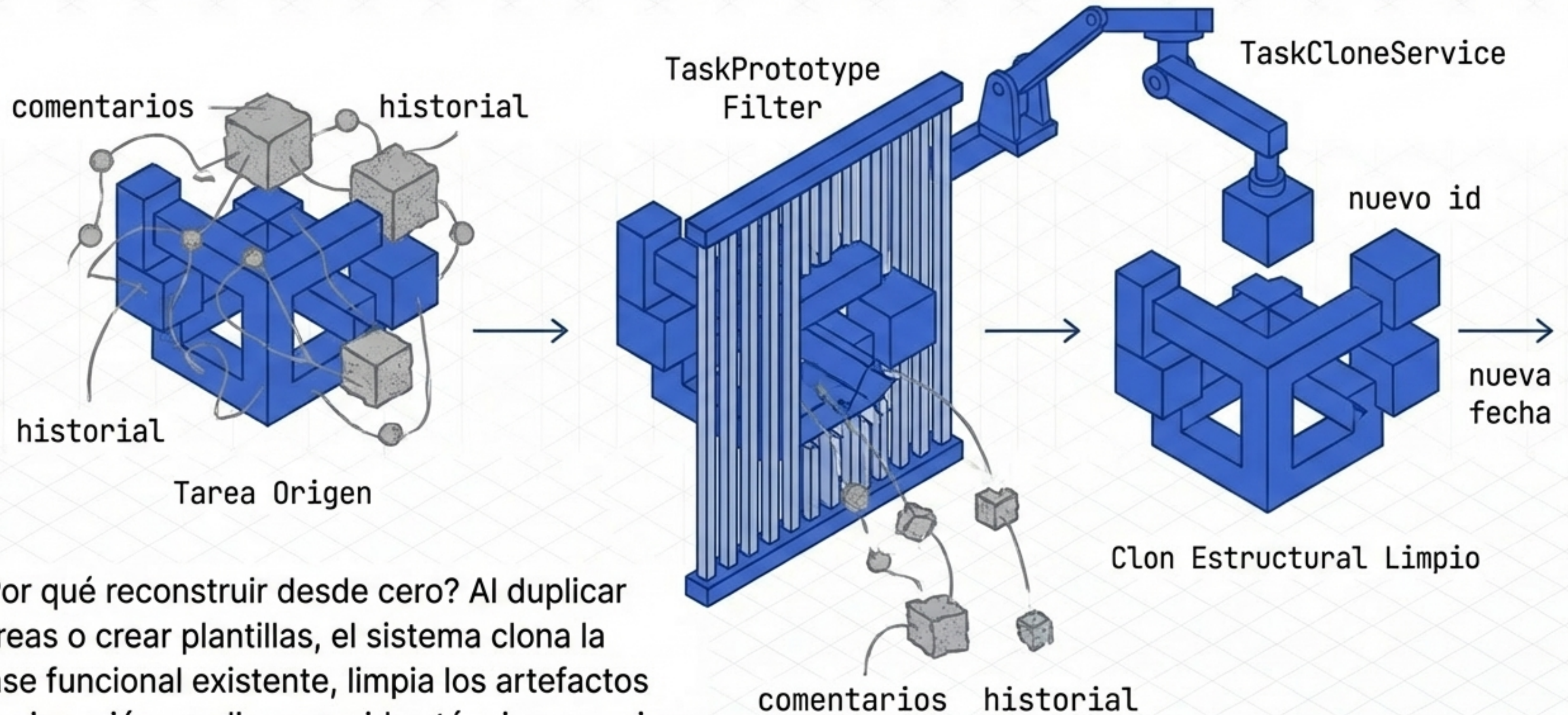


Factory Method: El Enrutador de Contexto

Delega la creación de la clase concreta según el contexto (tipo de tarea, perfil de usuario, canal de invitación). El servicio llamador solo conoce la interfaz abstracta, manteniendo el código base limpio y fácilmente extensible.



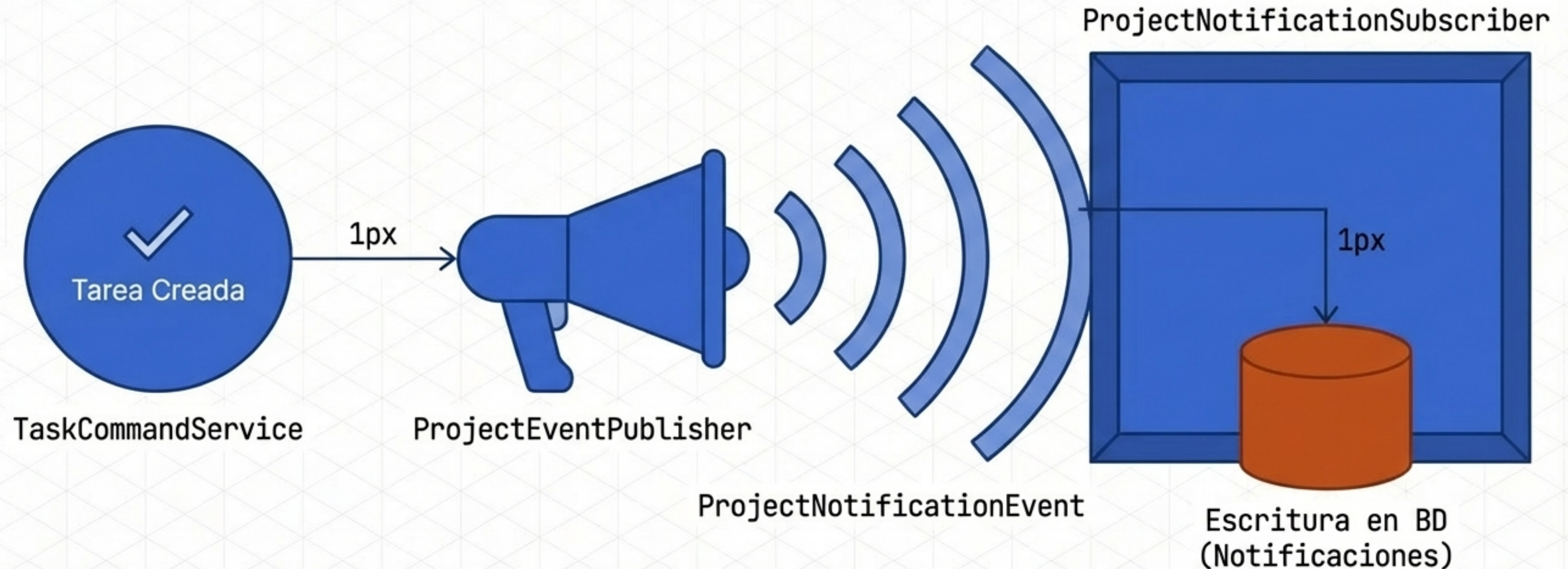
Prototype: La Bóveda de Clonación



¿Por qué reconstruir desde cero? Al duplicar tareas o crear plantillas, el sistema clona la base funcional existente, limpia los artefactos de ejecución y aplica overrides técnicos precisos.

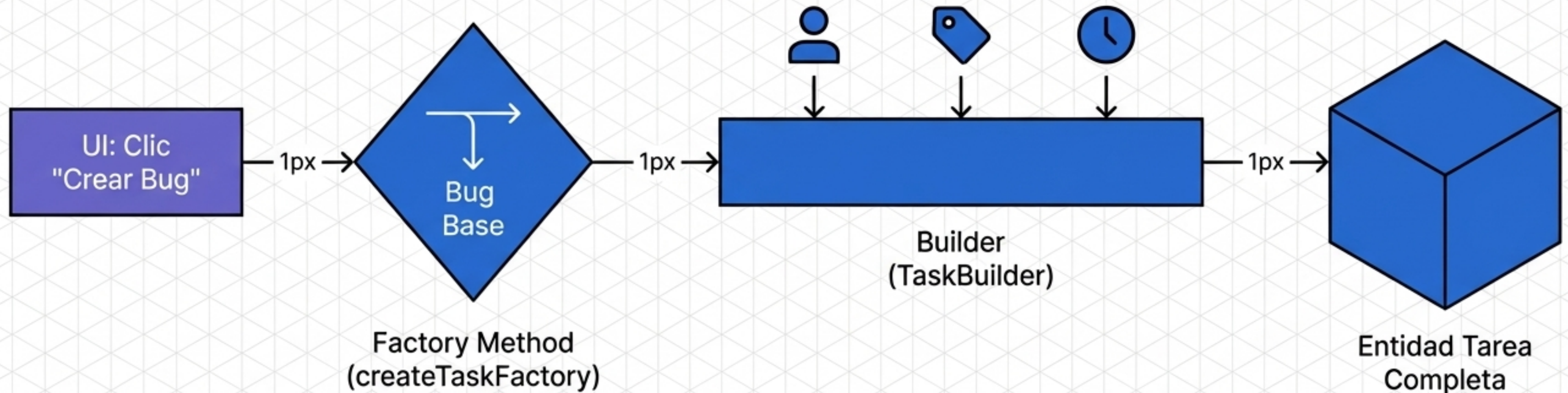
Observer: El Sistema de Transmisión

Desacopla de forma radical los comandos de dominio de la infraestructura de notificaciones. El servicio publica el hecho y termina su trabajo. No sabe (ni le importa) quién escucha o cómo se envían las alertas.



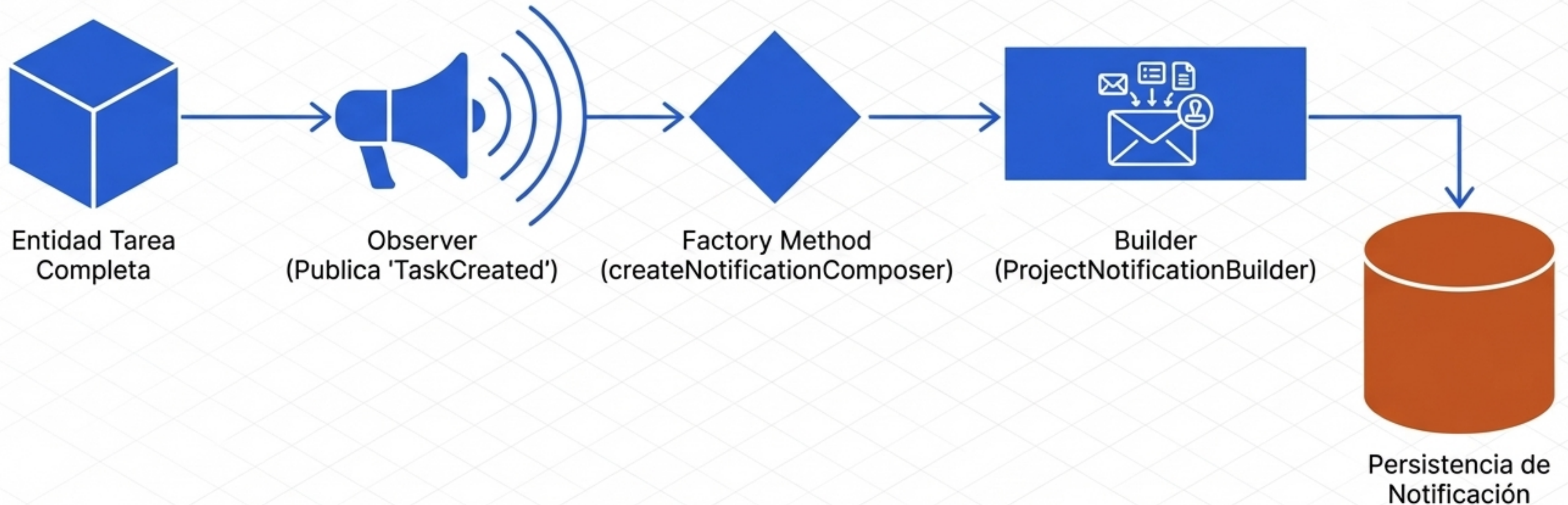
La Reacción en Cadena (Parte 1): Construcción

Los patrones operan como un motor cohesivo. Al crear una tarea, Factory Method asume el enrutamiento inicial basándose en el tipo, entregando la base a un Builder para que ensamble el agregado complejo paso a paso.



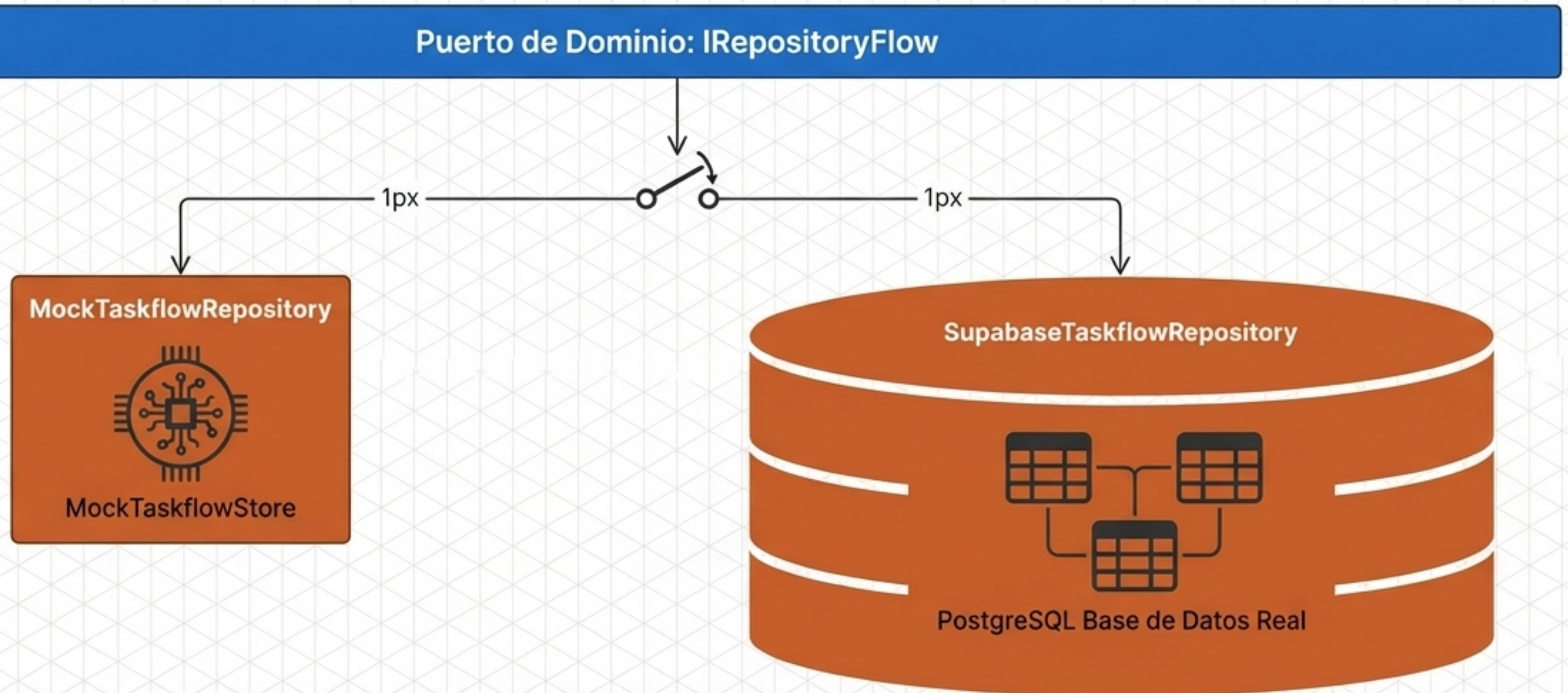
La Reacción en Cadena (Parte 2): Propagación

El motor no se detiene. El evento se propaga mediante Observer, lo que dispara una nueva fábrica y un nuevo constructor, dedicados exclusivamente a componer la notificación persistente sin bloquear el flujo original.



Aterrizaje en la Infraestructura: Supabase vs Mock

La infraestructura no define el patrón; simplemente lo ejecuta. Gracias a la inyección de dependencias, Taskflow alterna fluidamente entre un entorno real y un entorno Mock en memoria, sin alterar una sola línea del dominio.

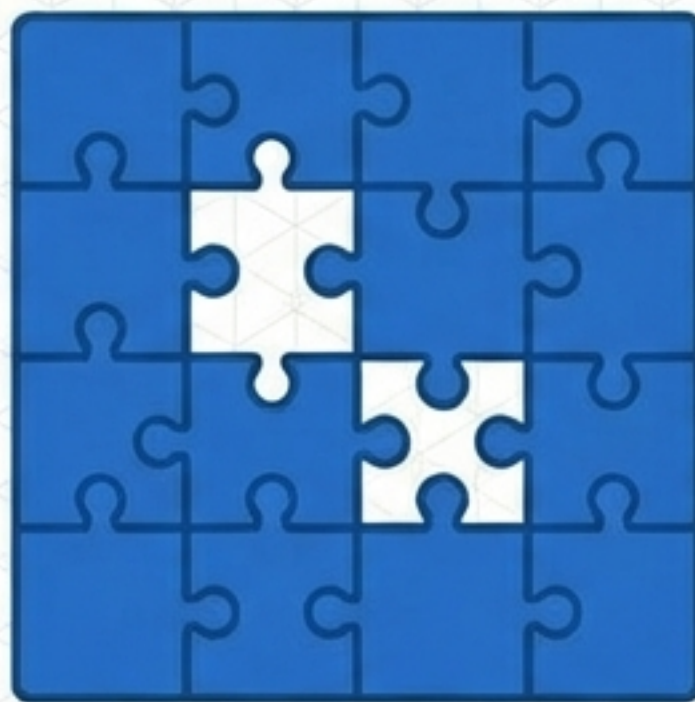


El Retorno de Inversión Arquitectónica



Responsabilidades Visibles

JetBrains Mono
Erradicación de condicionales gigantes y constructores opacos. Cada paso de la lógica tiene una dirección exacta.



Dominio Extensible

UI y lógica de negocio totalmente desacopladas. Agregar un nuevo tipo de tarea no rompe el código existente.



Trazabilidad Absoluta

Código auto-documentado, Código auto-documentado, ideal para evaluación académica y mantenimiento a largo plazo por equipos distribuidos.

Gracias